

Name : Md Shoeb Sardar  
ID : 0432310005101066 ( 7B )  
Course : Operating System Lab\  
Code : CSE 412

### Answer to the Q.N. (Lab Question 1)

```
#include <stdio.h>
#include <stdbool.h>

struct Process {
    int id, at, bt, ct, tat, wt;
    bool completed;
};

void solve_sjf() {
    int n, completed = 0, current_time = 0, total_idle_time = 0;
    float total_tat = 0, total_wt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];

    for (int i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("P%d Arrival Time & Burst Time: ", i + 1);
        scanf("%d %d", &p[i].at, &p[i].bt);
        p[i].completed = false;
    }

    printf("\nGantt Chart:\n");

    while (completed != n) {
        int idx = -1;
        int min_bt = 1e9;

        for (int i = 0; i < n; i++) {
            if (p[i].at <= current_time && !p[i].completed) {
                if (p[i].bt < min_bt) {
                    min_bt = p[i].bt;
                    idx = i;
                }
            }
        }
    }
}
```

```

if (idx != -1) {
    printf(" P%d (%d) |", p[idx].id, current_time + p[idx].bt);
    current_time += p[idx].bt;
    p[idx].ct = current_time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    p[idx].completed = true;
    completed++;
    total_tat += p[idx].tat;
    total_wt += p[idx].wt;
} else {
    printf(" IDLE (%d) |", current_time + 1);
    current_time++;
    total_idle_time++;
}
}

printf("\n\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
}

printf("\nAverage Turnaround Time: %.2f", total_tat / n);
printf("\nAverage Waiting Time: %.2f", total_wt / n);
printf("\nTotal CPU Idle Time: %d\n", total_idle_time);
}

int main() {
    solve_sjf();
    return 0;
}

```

```

4 struct Process {
5     int id, at, bt, ct, tat, wt;
6     bool completed;
7 };
8
9 void solve_sjf_priority() {
10     int n, completed = 0, current_time = 0, total_idle_time = 0;
11     float total_tat = 0, total_wt = 0;
12
13     printf("Enter number of processes: ");
14     scanf("%d", &n);
15     struct Process p[n];
16
17     for (int i = 0; i < n; i++) {
18         p[i].id = i + 1;
19         printf("P%d Arrival Time & Burst Time: ", i + 1);
20         scanf("%d %d", &p[i].at, &p[i].bt);
21         p[i].completed = false;
22     }
23
24     printf("\nGantt Chart:\n");
25
26     while (completed != n) {
27         int idx = -1;
28         int min_bt = 1e9;
29
30         for (int i = 0; i < n; i++) {
31             if (p[i].at <= current_time && !p[i].completed) {
32                 if (p[i].bt < min_bt) {
33                     min_bt = p[i].bt;
34                     idx = i;
35                 }
36             }
37         }
38
39         if (idx != -1) {
40             printf("P%d (%d) ", p[idx].id, current_time + p[idx].bt);
41             current_time += p[idx].bt;
42             p[idx].ct = current_time;
43             p[idx].tat = p[idx].ct - p[idx].at;
44             p[idx].wt = p[idx].tat - p[idx].bt;
45             p[idx].completed = true;
46             completed++;
47             total_tat += p[idx].tat;
48             total_wt += p[idx].wt;
49         } else {
50             printf("IDLE (%d) ", current_time + 1);
51             current_time++;
52             total_idle_time++;
53         }
54     }
55
56     printf("\n\nPID\tAT\tBT\tCT\tTAT\tWT\n");
57     for (int i = 0; i < n; i++) {
58         printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
59     }
60
61     printf("\nAverage Turnaround Time: %.2f", total_tat / n);
62 }

```

Enter number of processes: 5  
 P1 Arrival Time & Burst Time: 4 6  
 P2 Arrival Time & Burst Time: 2 3  
 P3 Arrival Time & Burst Time: 2 1  
 P4 Arrival Time & Burst Time: 2 3  
 P5 Arrival Time & Burst Time: 2 3

Gantt Chart:  
 | IDLE (1) | IDLE (2) | P3 (3) | P2 (6) | P4 (9) | P5 (12) | P1 (18) |

| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| 1   | 4  | 6  | 18 | 14  | 8  |
| 2   | 2  | 3  | 6  | 4   | 1  |
| 3   | 2  | 1  | 3  | 1   | 0  |
| 4   | 2  | 3  | 9  | 7   | 4  |
| 5   | 2  | 3  | 12 | 10  | 7  |

Average Turnaround Time: 7.20  
 Average Waiting Time: 4.00  
 Total CPU Idle Time: 2

## Answer to the Q.N. (Lab Question 2)

#include <stdio.h>

#include <stdbool.h>

struct Process {

int id, at, bt, priority, ct, tat, wt;

bool completed;

};

void solve\_sjf\_priority() {

int n, completed = 0, current\_time = 0, total\_idle\_time = 0;

float total\_tat = 0, total\_wt = 0;

```

printf("Enter number of processes: ");
scanf("%d", &n);
struct Process p[n];

for (int i = 0; i < n; i++) {
    p[i].id = i + 1;
    printf("P%d AT, BT, and Priority: ", i + 1);
    scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].priority);
    p[i].completed = false;
}

printf("\nGantt Chart:\n|");

while (completed != n) {
    int idx = -1;
    int min_bt = 1e9;
    int highest_pri = 1e9;

    for (int i = 0; i < n; i++) {
        if (p[i].at <= current_time && !p[i].completed) {
            if (p[i].bt < min_bt) {
                min_bt = p[i].bt;
                highest_pri = p[i].priority;
                idx = i;
            }
            else if (p[i].bt == min_bt) {
                if (p[i].priority < highest_pri) {
                    highest_pri = p[i].priority;
                    idx = i;
                }
            }
        }
    }
}

```

```

    if (idx != -1) {
        printf(" P%d (%d) |", p[idx].id, current_time + p[idx].bt);
        current_time += p[idx].bt;
        p[idx].ct = current_time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
        p[idx].completed = true;
        completed++;
        total_tat += p[idx].tat;
        total_wt += p[idx].wt;
    } else {
        printf(" IDLE (%d) |", current_time + 1);
        current_time++;
        total_idle_time++;
    }
}

printf("\n\nPID\tAT\tBT\tPRI\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].at, p[i].bt,
p[i].priority, p[i].ct, p[i].tat, p[i].wt);
}

printf("\nAverage Turnaround Time: %.2f", total_tat / n);
printf("\nAverage Waiting Time: %.2f", total_wt / n);
printf("\nTotal CPU Idle Time: %d\n", total_idle_time);
}

int main() {
    solve_sjf_priority();
    return 0;
}

```

```
lab2.c - Code::Blocks 17.12
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoryBlocks Settings Help

lab2.c lab2.c
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 struct Process {
5     int id, at, bt, priority, ct, tat, wt;
6     bool completed;
7 };
8
9 void solve_sjf_priority() {
10     int n, completed = 0, current_time = 0, total_idle_time = 0;
11     float total_tat = 0, total_wt = 0;
12     printf("Enter number of processes: ");
13     scanf("%d", &n);
14     struct Process p[n];
15
16     for (int i = 0; i < n; i++) {
17         p[i].id = i + 1;
18         printf("PID AT, BT, and Priority: ", i + 1);
19         scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].priority);
20         p[i].completed = false;
21     }
22
23     printf("\nGantt Chart:\n");
24
25     while (completed != n) {
26         int idx = -1;
27         int min_bt = 1e9;
28         int highest_pri = 1e9;
29
30         for (int i = 0; i < n; i++) {
31             if (p[i].at <= current_time && !p[i].completed) {
32                 if (p[i].bt < min_bt) {
33                     min_bt = p[i].bt;
34                     highest_pri = p[i].priority;
35                     idx = i;
36                 }
37             }
38             else if (p[i].bt == min_bt) {
39                 if (p[i].priority < highest_pri) {
40                     highest_pri = p[i].priority;
41                     idx = i;
42                 }
43             }
44         }
45     }
46 }
```

```

Enter number of processes: 3
P1 AT, BT, and Priority: 4 6 8
P2 AT, BT, and Priority: 4 6 8
P3 AT, BT, and Priority: 3 5 7

Gantt Chart:
| IDLE (1) | IDLE (2) | IDLE (3) | P3 (8) | P1 (14) | P2 (20) |

PID  AT  BT  PRI  CT  TAT  WT
1    4    6    8   14  10    4
2    4    6    8   20  16   10
3    3    5    7    8    5    0

Average Turnaround Time: 10.33
Average Waiting Time: 4.67
Total CPU Idle Time: 3

Process returned 0 (0x0)   execution time : 27.184 s
Press any key to continue.
```